Bits	Description	Type	Reset
7:5	**L0_NBITS**: Number of valid data bits for the lane 0 TMDS encoder, starting from bit 7 of the rotated data. Field values of 0 → 7 encode counts of 1 → 8 bits.	RW	0x0
4:0	**L0_ROT**: Right-rotate applied to the current shifter data before the lane 0 TMDS encoder.	RW	0x00

12.11.8. List of FIFO registers

The FIFO registers start at a base address of `0x50600000` (defined as HSTX_FIFO_BASE in the SDK).

Table 1258. List of HSTX_FIFO registers

Offset	Name	Info
0x0	STAT	FIFO status
0x4	FIFO	Write access to FIFO

HSTX_FIFO: STAT Register

Offset: 0x0

Description

FIFO status

Table 1259. STAT Register

Bits	Description	Type	Reset
31:11	Reserved.	-	-
10	**WOF**: FIFO was written when full. Write 1 to clear.	WC	0x0
9	**EMPTY**	RO	-
8	**FULL**	RO	-
7:0	**LEVEL**	RO	0x00

HSTX_FIFO: FIFO Register

Offset: 0x4

Table 1260. FIFO Register

Bits	Description	Type	Reset
31:0	Write access to FIFO	WF	0x00000000

12.12. TRNG

12.12.1. Overview

RP2350 contains an Arm IP-based True Random Number Generator block. It supports the following features:

- Compliance with FIPS Publication 140-2, BSI AIS-31, and NIST SP 800-90B
- Produces approximately 7.5 kb/s of entropy when the core runs at 150 MHz

On request, the TRNG block generates a block of 192 entropy bits generated by automatically processing a series of periodic samples from the TRNG block's internal Ring Oscillator (ROSC).

The TRNG block's ROSC is a free-running oscillator with no direct connection to the system clocks on RP2350. As a result, the ROSC generally runs asynchronously to the system clocks.

After a sufficient number of samples have been collected, the TRNG block completes the generation process and presents the random number in the `EHR_DATA[x]` result registers.

For more information, see ARM IP - True Random Number Generator

12.12.2. Configuration

The TRNG block contains three different built-in entropy checking mechanisms: At reset, these are all enabled by default and hence do not require explicit enabling.

You can configure the TRNG block in the following ways:

- Configure the frequency of the ROSC by selecting of one of four ROSC **chain lengths**, see TRNG_CONFIG.
- Configure the ROSC sampling period in terms of system clock ticks, see SAMPLE_CNT1.

Because the system clock generally runs much faster than the ROSC, the sampling period is expected to be at least a few tens of system clock ticks.

Because the characteristics of the TRNG ROSC and system clock frequency will differ for each implementation of the TRNG IP block, Arm details a TRNG characterisation procedure to determine the most appropriate ROSC chain length and sampling frequency settings on each SoC design. For details about that characterisation procedure, see ARM TrustZone True Number Generator.

Software drivers for the RP2350 TRNG block do not utilise the standard approach (see Section 12.12.4). As a result, software does not configure the ROSC length and sample count settings provided by the Arm characterisation procedure.

When configuring the TRNG block, consider the following principles:

- As average generation time increases, result quality increases and failed entropy checks decrease.
- A low sample count decreases average generation time, but increases the chance of NIST test-failing results and failed entropy checks.

For acceptable results with an average generation time of about 2 milliseconds, use ROSC chain length settings of 0 or 1 and sample count settings of 20-25.

Larger sample count settings (e.g. 100) provide proportionately slower average generation times. These settings significantly reduce, but do not eliminate NIST test failures and entropy check failures. Results occasionally take an especially long time to generate.

12.12.3. Operation

To initiate TRNG generation, set the `RND_SRC_EN` bit in RND_SOURCE_ENABLE. The TRNG will run until:

- It has successfully completed the generation of a random number.
- One, or more, of the internal entropy checking mechanisms indicates a failed run.

In either case, you can read the resultant status from RNG_ISR.

To generate TRNG block interrupts, set bits in RNG_IMR. Use RNG_ICR to clear active interrupt status bits.

The `EHR_DATA[x]` registers read 0 until successful generation has occurred, so the CPU cannot read random number results during generation,

After successful generation, read the last result register, `EHR_DATA[5]` to clear all of the result registers. If the result fails an entropy check, no results are presented and the `EHR_DATA[x]` registers all read as 0.

After TRNG generation and when not in use, the `RND_SRC_EN` bit should be cleared.

12.12.4. Caveats

The generation of random numbers by the TRNG block is not a deterministic process.

Although the modal and mean average times required to generate random numbers are quite similar, the generation process can occasionally take *much* longer to complete: in excess of 100 times the average. Any run resulting in a failed entropy check discards the result, requiring another generation process.

You can accommodate these unpredictable generation times in your system design. For example, you might generate a small pool of random numbers, initiating subsequent generation whenever space becomes available in the pool.

In the interests of simplicity and timing predictability, alternative approaches were adopted for the RP2350 bootrom and the SDK TRNG block drivers. The methodologies used can be found via the links below. However, nothing in the TRNG block in RP2350 precludes using the block as specified in Arm documentation.

12.12.4.1. Bootrom

The bootrom streams raw TRNG ROSC samples (the TRNG random source) directly into the hardware SHA-256 accelerator. It bypasses all internal checking and conditioning in the TRNG. SHA-256 is a robust hash function which avoids the pitfalls of some of the conditioning logic in the TRNG, most notably the von Neumann decorrelator.

The bootrom has some hard constraints which guide its implementation choices, most notably: *the bootrom must boot*. It cannot afford to poll the TRNG for an indeterminate amount of time to wait for a random number to appear. Complex error handling is also undesirable.

A link to the bootrom source can be found in Chapter 5. Consult the source code for the exact implementation of the per-boot random number generation, in `varm_boot_path.c`.

The A2 bootrom TRNG code is written in assembly due to various implementation constraints, and may not be that illuminating. The following is excerpted from the A1 bootrom source, lightly edited for readability:

```
// Boot RNG is derived by streaming a large number of TRNG ROSC samples
// into the SHA-256. BOOT_TRNG_SAMPLE_BLOCKS is the number of SHA-256
// blocks to hash, each containing 384 samples from the TRNG ROSC:
const int BOOT_TRNG_SAMPLE_BLOCKS = 25;

// Fixed delay is required after TRNG soft reset
trng_hw->trng_sw_reset = -1u;
(void)trng_hw->trng_sw_reset;
(void)trng_hw->trng_sw_reset;
// Initialise SHA internal state by writing START bit
sha256_hw->csr = SHA256_CSR_RESET | SHA256_CSR_START_BITS;

// Sample one ROSC bit into EHR every cycle, subject to CPU keeping up. More
// temporal resolution to measure ROSC phase noise is better, if we use a
// high quality hash function instead of naive VN decorrelation. (Also more
// metastability events, which are a secondary noise source)
trng_hw->sample_cnt1 = 0;
// Disable checks and bypass decorrelators, to stream raw TRNG ROSC samples:
trng_hw->trng_debug_control = -1u;
// Start ROSC if it is not already started
trng_hw->rnd_source_enable = -1u;
// Clear all interrupts (including EHR_VLD) -- we will check this
// later, after seeding RCP.
trng_hw->rng_icr = -1u;

// Each half-block (192 samples) takes approx 235 cycles, so 470 cycles/block:
for (int half_blocks = 0; half_blocks < 2 * BOOT_TRNG_SAMPLE_BLOCKS; ++half_blocks) {

    // Wait for 192 ROSC samples to fill EHR, this should take constant time:
    while (trng_hw->trng_busy)
```

```
        ;

    // Copy 6 EHR words to SHA-256, plus garbage (RND_SOURCE_ENABLE and
    // SAMPLE_CNT1) which pads us out to half of a SHA-256 block. This means
    // we can avoid checking SHA-256 ready whilst reading EHR, so we restart
    // sampling sooner. (SHA-256 becomes non-ready for 57 cycles after each
    // 16 words written.)
    io_ro_32 *src = &trng_hw->ehr_data[0];
    io_wo_32 *dst = &sha256_hw->wdata;
    for (int i = 0; i < 8; ++i) {
        *dst = src[i];
    }

    // TRNG is now sampling again, having started after we read the last EHR
    // word. Grab some in-progress SHA bits and use them to modulate the
    // chain length, to reduce chance of injection locking:
    trng_hw->trng_config = sha256_hw->sum[0];
}

// Wait for SHA result -- if skipped we get the previous block's digest. Note
// this never becomes true if we wrote a number of words % 16 != 0.
while (!(sha256_hw->csr & SHA256_CSR_SUM_VLD_BITS))
    ;

// The per-boot random will change on every core 0 reset (except debugger
// skipping ROM). If this is a problem then the user can sample the
// per-boot random into a preserved variable in main SRAM.
for (int i = 0; i < 4; ++i) {
    bootram->always.boot_random.e[i] = sha256_hw->sum[4 + i];
}

trng_hw->trng_config = 0;
// Stop ROSC as it's a waste of power
trng_hw->rnd_source_enable = 0;
```

The bootrom resets the SHA-256 and TRNG via RESETS immediately before the above code runs. This code typically runs with `clk_sys` running from the system ROSC, at its initial boot frequency of approximately 12 MHz. The 256-bit result is available in the SUM0 through SUM7 registers after the code completes.

This code does not represent best programming practice: for example it writes ones into reserved bits in the TRNG_DEBUG_CONTROL register. It was written with close reference to the hardware implementation. The above code listing serves only to document the *method* the bootrom uses to generate random numbers at boot time, for the once-per-boot random number available via the `get_sys_info()` ROM API as well as for initialising the RCP salt registers (Section 3.6.3.1).

12.12.4.2. SDK

The `pico_rand` library uses the TRNG as one of its entropy sources. It streams raw ROSC samples from the TRNG ROSC in a similar manner to the bootrom. It uses the `xoroshiro128**` and `splitmix64()` PRNG functions to condition the output.

12.12.5. List of registers

The TRNG control registers start at a base address of `0x400f0000` (defined as TRNG_BASE in the SDK).

Table 1261. List of TRNG registers

Offset	Name	Info
0x100	RNG_IMR	Interrupt masking.

Offset	Name	Info
0x104	RNG_ISR	RNG status register. If corresponding RNG_IMR bit is unmasked, an interrupt will be generated.
0x108	RNG_ICR	Interrupt/status bit clear Register.
0x10c	TRNG_CONFIG	Selecting the inverter-chain length.
0x110	TRNG_VALID	192 bit collection indication.
0x114	EHR_DATA0	RNG collected bits.
0x118	EHR_DATA1	RNG collected bits.
0x11c	EHR_DATA2	RNG collected bits.
0x120	EHR_DATA3	RNG collected bits.
0x124	EHR_DATA4	RNG collected bits.
0x128	EHR_DATA5	RNG collected bits.
0x12c	RND_SOURCE_ENABLE	Enable signal for the random source.
0x130	SAMPLE_CNT1	Counts clocks between sampling of random bit.
0x134	AUTOCORR_STATISTIC	Statistics about autocorrelation test activations.
0x138	TRNG_DEBUG_CONTROL	Debug register.
0x140	TRNG_SW_RESET	Generate internal SW reset within the RNG block.
0x1b4	RNG_DEBUG_EN_INPUT	Enable the RNG debug mode
0x1b8	TRNG_BUSY	RNG Busy indication.
0x1bc	RST_BITS_COUNTER	Reset the counter of collected bits in the RNG.
0x1c0	RNG_VERSION	Displays the version settings of the TRNG.
0x1e0	RNG_BIST_CNTR_0	Collected BIST results.
0x1e4	RNG_BIST_CNTR_1	Collected BIST results.
0x1e8	RNG_BIST_CNTR_2	Collected BIST results.

TRNG: RNG_IMR Register

Offset: 0x100

Description

Interrupt masking.

Table 1262. RNG_IMR Register

Bits	Description	Type	Reset
31:4	Reserved.	-	-
3	**VN_ERR_INT_MASK**: Set to 1 to mask (disable) this interrupt: no interrupt will be generated. See RNG_ISR for an explanation on this interrupt.	RW	0x1
2	**CRNGT_ERR_INT_MASK**: Set to 1 to mask (disable) this interrupt: no interrupt will be generated. See RNG_ISR for an explanation on this interrupt.	RW	0x1
1	**AUTOCORR_ERR_INT_MASK**: Set to 1 to mask (disable) this interrupt: no interrupt will be generated. See RNG_ISR for an explanation on this interrupt.	RW	0x1
0	**EHR_VALID_INT_MASK**: Set to 1 to mask (disable) this interrupt: no interrupt will be generated. See RNG_ISR for an explanation on this interrupt.	RW	0x1

TRNG: RNG_ISR Register

Offset: 0x104

Description

RNG status register. If corresponding RNG_IMR bit is unmasked, an interrupt will be generated.

Table 1263. RNG_ISR Register

Bits	Description	Type	Reset
31:4	Reserved.	-	-
3	**VN_ERR**: 1 indicates von Neumann error. Error in von Neumann occurs if 32 consecutive collected bits are identical, ZERO or ONE.	RO	0x0
2	**CRNGT_ERR**: 1 indicates CRNGT in the RNG test failed. Failure occurs when two consecutive blocks of 16 collected bits are equal.	RO	0x0
1	**AUTOCORR_ERR**: 1 indicates Autocorrelation test failed four times in a row. When set, RNG ceases functioning until next reset.	RO	0x0
0	**EHR_VALID**: 1 indicates that 192 bits have been collected in the RNG, and are ready to be read.	RO	0x0

TRNG: RNG_ICR Register

Offset: 0x108

Description

Interrupt/status bit clear Register.

Table 1264. RNG_ICR Register

Bits	Description	Type	Reset
31:4	Reserved.	-	-
3	**VN_ERR**: Write 1 to clear corresponding bit in RNG_ISR.	RW	0x0
2	**CRNGT_ERR**: Write 1 to clear corresponding bit in RNG_ISR.	RW	0x0
1	**AUTOCORR_ERR**: Cannot be cleared by SW! Only RNG reset clears this bit.	RW	0x0
0	**EHR_VALID**: Write 1 - clear corresponding bit in RNG_ISR.	RW	0x0

TRNG: TRNG_CONFIG Register

Offset: 0x10c

Description

Selecting the inverter-chain length.

Table 1265. TRNG_CONFIG Register

Bits	Description	Type	Reset
31:2	Reserved.	-	-
1:0	**RND_SRC_SEL**: Selects the number of inverters (out of four possible selections) in the ring oscillator (the entropy source). Higher values select longer inverter chain lengths.	RW	0x0

TRNG: TRNG_VALID Register

Offset: 0x110

Description

192 bit collection indication.

Table 1266. TRNG_VALID Register

Bits	Description	Type	Reset
31:1	Reserved.	-	-
0	**EHR_VALID**: 1 indicates that collection of bits in the RNG is completed, and data can be read from EHR_DATA register.	RO	0x0

TRNG: EHR_DATA0, EHR_DATA1, ..., EHR_DATA4, EHR_DATA5 Registers

Offsets: 0x114, 0x118, ..., 0x124, 0x128

Description

RNG collected bits.

Table 1267. EHR_DATA0, EHR_DATA1, ..., EHR_DATA4, EHR_DATA5 Registers

Bits	Description	Type	Reset
31:0	Bits [(32*(i+1))-1:(32*i)] of Entropy Holding Register.	RO	0x00000000

TRNG: RND_SOURCE_ENABLE Register

Offset: 0x12c

Description

Enable signal for the random source.

Table 1268. RND_SOURCE_ENABLE Register

Bits	Description	Type	Reset
31:1	Reserved.	-	-
0	**RND_SRC_EN**: * 1 - entropy source is enabled. * 0 - entropy source is disabled	RW	0x0

TRNG: SAMPLE_CNT1 Register

Offset: 0x130

Description

Counts clocks between sampling of random bit.

Table 1269. SAMPLE_CNT1 Register

Bits	Description	Type	Reset
31:0	**SAMPLE_CNTR1**: Sets the number of rng_clk cycles between two consecutive ring oscillator samples. Note: If the von Neumann decorrelator is bypassed, the minimum value for sample counter must not be less than seventeen	RW	0x0000ffff

TRNG: AUTOCORR_STATISTIC Register

Offset: 0x134

Description

Statistics about autocorrelation test activations.

Table 1270. AUTOCORR_STATISTIC Register

Bits	Description	Type	Reset
31:22	Reserved.	-	-
21:14	**AUTOCORR_FAILS**: Count each time an autocorrelation test fails. Any write to the register reset the counter. Stop collecting statistic if one of the counters reached the limit.	RW	0x00

Bits	Description	Type	Reset
13:0	**AUTOCORR_TRYS**: Count each time an autocorrelation test starts. Any write to the register reset the counter. Stop collecting statistic if one of the counters reached the limit.	RW	0x0000

TRNG: TRNG_DEBUG_CONTROL Register

Offset: 0x138

Description

Debug register.

Table 1271. TRNG_DEBUG_CONTROL Register

Bits	Description	Type	Reset
31:4	Reserved.	-	-
3	**AUTO_CORRELATE_BYPASS**: When set, the autocorrelation test in the TRNG module is bypassed.	RW	0x0
2	**TRNG_CRNGT_BYPASS**: When set, the CRNGT test in the RNG is bypassed.	RW	0x0
1	**VNC_BYPASS**: When set, the Von-Neuman balancer is bypassed (including the 32 consecutive bits test). N/A	RW	0x0
0	Reserved.	-	-

TRNG: TRNG_SW_RESET Register

Offset: 0x140

Description

Generate internal SW reset within the RNG block.

Table 1272. TRNG_SW_RESET Register

Bits	Description	Type	Reset
31:1	Reserved.	-	-
0	**TRNG_SW_RESET**: Writing 1 to this register causes an internal RNG reset.	RW	0x0

TRNG: RNG_DEBUG_EN_INPUT Register

Offset: 0x1b4

Description

Enable the RNG debug mode

Table 1273. RNG_DEBUG_EN_INPUT Register

Bits	Description	Type	Reset
31:1	Reserved.	-	-
0	**RNG_DEBUG_EN**: * 1 - debug mode is enabled. * 0 - debug mode is disabled	RW	0x0

TRNG: TRNG_BUSY Register

Offset: 0x1b8

Description

RNG Busy indication.

Table 1274. TRNG_BUSY Register

Bits	Description	Type	Reset
31:1	Reserved.	-	-
0	**TRNG_BUSY**: Reflects rng_busy status.	RO	0x0

TRNG: RST_BITS_COUNTER Register

Offset: 0x1bc

Description

Reset the counter of collected bits in the RNG.

Table 1275. RST_BITS_COUNTER Register

Bits	Description	Type	Reset
31:1	Reserved.	-	-
0	**RST_BITS_COUNTER**: Writing any value to this address will reset the bits counter and RNG valid registers. RND_SORCE_ENABLE register must be unset in order for the reset to take place.	RW	0x0

TRNG: RNG_VERSION Register

Offset: 0x1c0

Description

Displays the version settings of the TRNG.

Table 1276. RNG_VERSION Register

Bits	Description	Type	Reset
31:8	Reserved.	-	-
7	**RNG_USE_5_SBOXES**: * 1 - 5 SBOX AES. * 0 - 20 SBOX AES	RO	0x0
6	**RESEEDING_EXISTS**: * 1 - Exists. * 0 - Does not exist	RO	0x0
5	**KAT_EXISTS**: * 1 - Exists. * 0 - Does not exist	RO	0x0
4	**PRNG_EXISTS**: * 1 - Exists. * 0 - Does not exist	RO	0x0
3	**TRNG_TESTS_BYPASS_EN**: * 1 - Exists. * 0 - Does not exist	RO	0x0
2	**AUTOCORR_EXISTS**: * 1 - Exists. * 0 - Does not exist	RO	0x0
1	**CRNGT_EXISTS**: * 1 - Exists. * 0 - Does not exist	RO	0x0
0	**EHR_WIDTH_192**: * 1 - 192-bit EHR. * 0 - 128-bit EHR	RO	0x0

TRNG: RNG_BIST_CNTR_0 Register

Offset: 0x1e0

Description

Collected BIST results.

Table 1277. RNG_BIST_CNTR_0 Register

Bits	Description	Type	Reset
31:22	Reserved.	-	-
21:0	**ROSC_CNTR_VAL**: Reflects the results of RNG BIST counter.	RO	0x000000

TRNG: RNG_BIST_CNTR_1 Register

Offset: 0x1e4

Description

Collected BIST results.

Table 1278. RNG_BIST_CNTR_1 Register

Bits	Description	Type	Reset
31:22	Reserved.	-	-
21:0	**ROSC_CNTR_VAL**: Reflects the results of RNG BIST counter.	RO	0x000000

TRNG: RNG_BIST_CNTR_2 Register

Offset: 0x1e8

Description

Collected BIST results.

Table 1279. RNG_BIST_CNTR_2 Register

Bits	Description	Type	Reset
31:22	Reserved.	-	-
21:0	**ROSC_CNTR_VAL**: Reflects the results of RNG BIST counter.	RO	0x000000

12.13. SHA-256 accelerator

RP2350 is equipped with an implementation of the SHA-256 hash algorithm, as defined in the FIPS 180-4 standard available from NIST publications. A hash algorithm digests an arbitrary-length stream of data, known as the **message**, and produces a fixed-size result, known as a **hash**. In the case of SHA-256, the result is always 256 bits in size. Hash algorithms are designed such that:

- Given the hash, it is impossible (or implausibly computationally hard) to recover the original message.
- Small changes to the original message result, on average, in large changes to the hash.
- Given a message with a particular hash, it is impossible (or implausibly computationally hard) to generate a different message with the same hash.

These properties make hash algorithms useful for checking the integrity of data, in the face of both accidental bit flips and deliberate tampering.

To compute a SHA-256 with the RP2350 SHA-256 accelerator:

1. Initialise the algorithm state by writing a 1 to CSR.START.
2. Write the message to the WDATA register, polling CSR.WDATA_RDY in between writes.

3. Write additional trailer and padding data to WDATA, as described in Section 12.13.1 below.
4. Poll CSR.SUM_VLD to wait for the last block to be digested.
5. Read the 256-bit result from the 8 read-only result registers starting at SUM0.

12.13.1. Message padding

Pad message content according to the standard SHA-256 method as described in the FIPS 180-4 Secure Hash Standard: append the message with single bit `1`, then a number of `0` bits, then a 64-bit count of the number of message bits. So for a message `M` with length `L` bits the padded message should be:

1. message `M`
2. `1`
3. `k` zero bits, where `k` is the smallest non-negative solution to the equation: `L` + 1 + `k` = 448 mod 512
4. a 64-bit block indicating `L` (the length of the message) in binary

For example, the 8 bit ASCII message `abc` has a length of 24 bits. This is padded with `1`, then 448-(24+1) = 423 `0` bits, and then the message length as a 64-bit value as follows:

```
01100001 01100010 01100011 1 00000000 000...0 00000000 000...0 00011000
|---------message--------| 1 |--423 0 bits--| |------64 bit len-------|
```

12.13.2. Throughput

SHA-256 processes data one 512-bit block at a time. This requires 16 32-bit writes, 32 16-bit writes, or 64 8-bit writes to the WDATA register. An APB register write costs 4 cycles, so it takes at least 64 system clock cycles to write a data block.

Once a full block is transferred, the SHA core takes a further 57 cycles to complete the block digest. CSR.WDATA_RDY goes low, and you must not write to WDATA during this time.

The maximum throughput is therefore one block per 121 system clock cycles, or 0.53 bytes per cycle. At a 150 MHz system clock this is 79.3 MB/s. This throughput is achieved when you use 32-bit transfers from the DMA. Using narrower transfers result in lower throughput, as does polling the CSR.WDATA_RDY flag when transferring data from the processor.

12.13.3. Data size and endianness

Data is sent in message blocks of 512 bits, padded as described in Section 12.13.1. The SHA-256 accelerator updates its 256-bit output state for each input block. The SHA-256 algorithm is defined in terms of big-endian message words, but this accelerator provides a byte swap function via CSR.BSWAP to support little-endian data. `BSWAP` is set by default. For more information, see the register descriptions.

WDATA supports 8-bit, 16-bit and 32-bit writes. The bus interface accumulates 8 and 16-bit writes in a 32-bit shift register before passing them into the SHA-256 algorithm core. This means you must take care when mixing writes of different sizes, because taking the shift register level from less than to greater than 32 bits in a single write will silently drop data. You can avoid this issue by not mixing WDATA write sizes within a single SHA-256 message block (64 bytes).

12.13.4. DMA DREQ interface

The block can request the DMA controller to send entire blocks of data at once. Configure transfer size using

CSR.DMA_SIZE so that the DMA controller requests the correct number of transfers.

The DREQ always requests one full SHA block of data at a time. Do not start a DMA on a non-block boundary.

12.13.5. List of registers

The SHA-256 registers start at a base address of `0x400f8000` (defined as SHA256_BASE in SDK).

Table 1280. List of SHA256 registers

Offset	Name	Info
0x00	CSR	Control and status register
0x04	WDATA	Write data register
0x08	SUM0	256-bit checksum result. Contents are undefined when CSR_SUM_VLD is 0.
0x0c	SUM1	256-bit checksum result. Contents are undefined when CSR_SUM_VLD is 0.
0x10	SUM2	256-bit checksum result. Contents are undefined when CSR_SUM_VLD is 0.
0x14	SUM3	256-bit checksum result. Contents are undefined when CSR_SUM_VLD is 0.
0x18	SUM4	256-bit checksum result. Contents are undefined when CSR_SUM_VLD is 0.
0x1c	SUM5	256-bit checksum result. Contents are undefined when CSR_SUM_VLD is 0.
0x20	SUM6	256-bit checksum result. Contents are undefined when CSR_SUM_VLD is 0.
0x24	SUM7	256-bit checksum result. Contents are undefined when CSR_SUM_VLD is 0.

SHA256: CSR Register

Offset: 0x00

Description

Control and status register

Table 1281. CSR Register

Bits	Description	Type	Reset
31:13	Reserved.	-	-

Bits	Description	Type	Reset
12	**BSWAP**: Enable byte swapping of 32-bit values at the point they are committed to the SHA message scheduler. This block's bus interface assembles byte/halfword data into message words in little-endian order, so that DMAing the same buffer with different transfer sizes always gives the same result on a little-endian system like RP2350. However, when marshalling bytes into blocks, SHA expects that the first byte is the **most significant** in each message word. To resolve this, once the bus interface has accumulated 32 bits of data (either a word write, two halfword writes in little-endian order, or four byte writes in little-endian order) the final value can be byte-swapped before passing to the actual SHA core. This feature is enabled by default because using the SHA core to checksum byte buffers is expected to be more common than having preformatted SHA message words lying around.	RW	0x1
11:10	Reserved.	-	-
9:8	**DMA_SIZE**: Configure DREQ logic for the correct DMA data size. Must be configured before the DMA channel is triggered. The SHA-256 core's DREQ logic requests one entire block of data at once, since there is no FIFO, and data goes straight into the core's message schedule and digest hardware. Therefore, when transferring data with DMA, CSR_DMA_SIZE must be configured in advance so that the correct number of transfers can be requested per block.	RW	0x2
	Enumerated values:		
	0x0 → 8BIT		
	0x1 → 16BIT		
	0x2 → 32BIT		
7:5	Reserved.	-	-
4	**ERR_WDATA_NOT_RDY**: Set when a write occurs whilst the SHA-256 core is not ready for data (WDATA_RDY is low). Write one to clear.	WC	0x0
3	Reserved.	-	-
2	**SUM_VLD**: If 1, the SHA-256 checksum presented in registers SUM0 through SUM7 is currently valid. Goes low when WDATA is first written, then returns high once 16 words have been written and the digest of the current 512-bit block has subsequently completed.	RO	0x1
1	**WDATA_RDY**: If 1, the SHA-256 core is ready to accept more data through the WDATA register. After writing 16 words, this flag will go low for 57 cycles whilst the core completes its digest.	RO	0x1

Bits	Description	Type	Reset
0	**START**: Write 1 to prepare the SHA-256 core for a new checksum. The SUMx registers are initialised to the proper values (fractional bits of square roots of first 8 primes) and internal counters are cleared. This immediately forces WDATA_RDY and SUM_VLD high. START must be written before initiating a DMA transfer to the SHA-256 core, because the core will always request 16 transfers at a time (1 512-bit block). Additionally, the DMA channel should be configured for a multiple of 16 32-bit transfers.	SC	0x0

SHA256: WDATA Register

Offset: 0x04

Description

Write data register

Table 1282. WDATA Register

Bits	Description	Type	Reset
31:0	After pulsing START and writing 16 words of data to this register, WDATA_RDY will go low and the SHA-256 core will complete the digest of the current 512-bit block. Software is responsible for ensuring the data is correctly padded and terminated to a whole number of 512-bit blocks. After this, WDATA_RDY will return high, and more data can be written (if any). This register supports word, halfword and byte writes, so that DMA from non-word-aligned buffers can be supported. The total amount of data per block remains the same (16 words, 32 halfwords or 64 bytes) and byte/halfword transfers must not be mixed within a block.	WF	0x00000000

SHA256: SUM0 Register

Offset: 0x08

Table 1283. SUM0 Register

Bits	Description	Type	Reset
31:0	256-bit checksum result. Contents are undefined when CSR_SUM_VLD is 0.	RO	0x00000000

SHA256: SUM1 Register

Offset: 0x0c

Table 1284. SUM1 Register

Bits	Description	Type	Reset
31:0	256-bit checksum result. Contents are undefined when CSR_SUM_VLD is 0.	RO	0x00000000

SHA256: SUM2 Register

Offset: 0x10